

Frustration-Risk Score — Q&A

How it's calculated, where its inputs come from, and what the tutor actually does with it. Source: `app/ml/multimodal_fusion.py`, `app/ml/agents/pedagogical_agent.py`, `app/ml/agents/content_agent.py`, `app/ml/agents/diagnostic_agent.py`

Q1. What is frustration risk?

A single number from 0 to 1, computed fresh on every tutoring turn, estimating “**is this student about to give up**” — not “is this student unhappy.” It is a rule-based proxy built from evidence of struggle (bad performance, low mastery, uncertain speech, and optionally facial emotion), not a direct measurement of feelings.

Q2. What signals go into it — where does it get its data from?

Three independent modalities are combined by `MultimodalFusion.fuse()`, each with a fixed reliability weight used elsewhere in the fusion (BKT is always available, camera/mic only when active):

Signal	Source	Availability
BKT mastery	Bayesian Knowledge Tracing — specifically <code>bkt_min</code> , the learner's <i>weakest</i> skill, not the average	Always available
Recent accuracy	Fraction correct over the last ~10 questions answered	Always available
Audio confidence	Speech-to-text analysis confidence from the microphone	Only when mic is used
Facial emotion	DeepFace CNN classification of webcam frames — blended in separately, see Q4	Only when camera is on

Q3. How exactly is it calculated — the formula?

It's a points system, not a weighted average. Four rules each independently add a fixed amount to a running total (capped at 1.0), from `multimodal_fusion.py`:

```
frustration_risk = 0.0
if recent_accuracy < 0.4:    frustration_risk += 0.3
if bkt_min < 0.3:           frustration_risk += 0.3
if audio_confidence < 0.3:  frustration_risk += 0.2 (only if mic is on)
if attempts > 5 and recent_accuracy < 0.3: frustration_risk += 0.2
frustration_risk = min(1.0, frustration_risk)
```

Why 0.3 / 0.3 / 0.2 / 0.2? The four weights sum to exactly 1.0 by design: if every warning sign fires at once, the score lands exactly at the ceiling. The two **0.3** weights go to the most direct, always-available, hardware-independent signals (current accuracy and the weakest-skill BKT mastery). The two **0.2** weights go to weaker or more conditional evidence: audio confidence is only a *proxy* for uncertainty (could just be a quiet mic), and the “persistence” rule is largely redundant with the accuracy rule already firing — so it adds less new information. **Why the minimum skill, not the average?** A student with five skills at 90% and one stuck at 10% would look fine on average (~75%) — using the minimum catches the one skill actually causing frustration instead of smoothing it away.

Note: these exact weights are a hand-tuned heuristic in the code, not a value derived from a study or trained on data.

Q4. Is that the whole calculation, or does anything else feed into it?

One more step happens afterward, in `diagnostic_agent.py`: if the camera is on and DeepFace produced a real facial-emotion frustration score, it gets blended into the number above at a fixed ratio:

```
if emotion_frustration_score > 0:  
    frustration_risk = frustration_risk × 0.6 + emotion_frustration_score × 0.4
```

So the behavioural/performance score (accuracy + mastery + voice) always carries the majority weight (60%), with facial emotion capped at a 40% influence — camera data adjusts the score, it never fully overrides it on its own at this stage.

Q5. What is it used for — what does the system actually do with it?

Three concrete, escalating uses — not just a number shown on a dashboard:

Trigger	What fires	Effect
frustration_risk ≥ 0.75	PedagogicalAgent forces TeachingAction.ENCOURAGE, overriding whatever the normal mastery/ZPD logic would have picked	Hard override — runs before any other teaching decision
frustration_risk 0.40–0.75 (with a real camera/mic signal)	An “IMPORTANT” alert line is inserted into the LLM's system prompt	Softer nudge — tells the model the student seems to be struggling, without forcing a specific action
frustration_risk < 0.40	No override, no alert	Normal teaching action proceeds (explain / hint / advance / remediate as usual)

Q6. When it overrides the tutor, what does the student actually see change?

The exact instruction injected into the LLM's system prompt when ENCOURAGE fires (from `pedagogical_agent.py`):

“ENGAGEMENT ALERT: frustration risk 82%, overall engagement 31%... The student is showing signs of frustration or disengagement. **DO NOT ask a new question or add complexity.** Acknowledge their effort warmly, normalise the difficulty, and offer ONE tiny, achievable next step. Use encouraging language. Keep it short and supportive.”

Three concrete behaviour changes result: **(1) pacing freezes** — no new questions or harder material, regardless of what mastery/ZPD would otherwise recommend; **(2) scope shrinks** — the response is limited to one small, achievable step instead of a full explanation; **(3) tone shifts** — the model is directed to validate the student emotionally before anything academic, and to keep it brief.

There is also a stronger tier, used only when the *camera* directly detects real distress (angry / disgust / fear / sad ≥ 70% via DeepFace), which is even more restrictive: **“DO NOT continue with new content.** Acknowledge the difficulty with empathy, validate their feelings...” — this blocks new academic content entirely, not just new questions.